

# Beyond Vibe Coding: A Goal-to-Milestone Framework for AI-Supported Creative Programming in K–12

Sichuang FAN

*Faculty of Education, Western University, Canada*

**Abstract:** AI coding tools can help young learners create software from natural-language prompts, but their dominant interaction pattern often optimizes for code production rather than learner-owned progression. This short paper argues that, for K–12 creative programming, the central design problem is how to preserve the motivation of a personally meaningful project while turning that project into a learning process. Drawing on constructionist computing education, non-formal K–12 computing, self-determination theory, scaffolding, cognitive load theory, metacognition, and emerging work on AI-supported programming, we propose a Goal-to-Milestone framework. The framework treats backward path planning and the next bounded milestone as the primary units of AI support. After a learner names an idea, the learner and AI co-construct a project flowchart, make design judgments, choose an accessible first milestone, articulate done criteria, sketch logic, build, preview, debug from observed behavior, receive a mini-explanation, and select the next milestone. We do not report empirical outcomes; instead, we offer a literature-grounded position, two compressed design scenarios, and a conjecture map for future research.

**Keywords:** K–12 computing education, AI-assisted programming, creative coding, metacognition, scaffolding

## 1 Introduction

AI coding tools are changing what young learners can plausibly make. They can generate code, explain errors, and help novices move from an idea to a working artifact (Kazemitabaar et al., 2023). Emerging studies of vibe coding also show that prompt-driven software creation can alter how learners seek help, inspect code, and understand the systems they are making (Si et al., 2026; Thorgeirsson et al., 2026). For K–12 creative programming, however, the key question is not simply whether AI can produce working code. The harder question is how AI support can help learners turn a personally meaningful idea into a project path that remains understandable, revisable, and learner-owned.

This paper focuses on upper-elementary and middle-school learners in hybrid learning spaces, such as after-school programs, clubs, libraries, homes, maker spaces, online communities, and classrooms where projects are only partly shaped by formal assignments. Youth programming has long been organized around personally meaningful making, sharing, remixing, and computational participation (Brennan & Resnick, 2012; Kafai & Burke, 2014; Papert, 1980). If a learner begins with a vague goal such as “make a quiz game about my school,” the design challenge is finding the fine line between protecting the motivation that comes from wanting to finish a real project and helping the learner reason backward from that imagined artifact into learnable design decisions, criteria, milestones, and explanations. We therefore propose the Goal-to-Milestone framework as a pre-code and between-milestone interaction layer.

## 2 Background and Design Gap

The literature motivates a narrow design gap. Constructionist and computational participation perspectives suggest that AI support for youth programming should preserve opportunities for learners to plan, explain, debug, and make choices that feel like their own (Brennan & Resnick, 2012; Kafai & Burke, 2014; Papert, 1980). Non-formal K–12 computing settings make this especially important because personalization, relevance, fun, identity, and community are central (Gardner et al., 2022). Self-determination theory further suggests that a personally meaningful idea can support autonomy, while visible progress can support competence (Deci & Ryan, 2000).

At the same time, motivation does not remove the need for structure. Scaffolding frames tutoring as temporary support for tasks learners could not yet complete alone (Wood et al., 1976), and cognitive load theory cautions that unguided complex problem solving can consume attention that might otherwise support learning (Sweller, 1988). Recent AI programming systems show the same tension: code generators can help novices create code, while CodeAid and Stitch emphasize explanations, subgoals, fix suggestions, and stepwise tutoring rather than complete answers (Kazemitabaar et al., 2024; Si et al., 2025). What remains underdesigned is the interaction layer that helps a learner move from idea to code without making the AI the hidden planner.

We argue that this missing layer should be designed as metacognitive scaffolding. Metacognition involves knowledge about cognition and monitoring of cognitive activity (Flavell, 1979), and self-regulated learning emphasizes planning, monitoring, and reflection (Zimmerman, 2002). Programming-specific work also suggests that metacognitive scaffolding can support problem-solving activity (Shin et al., 2023). In our framework, these practices are not added as after-the-fact reflection prompts. They are embedded in the project path itself: planning a flowchart, choosing a milestone, stating done criteria, comparing preview behavior with expectations, and explaining what changed.

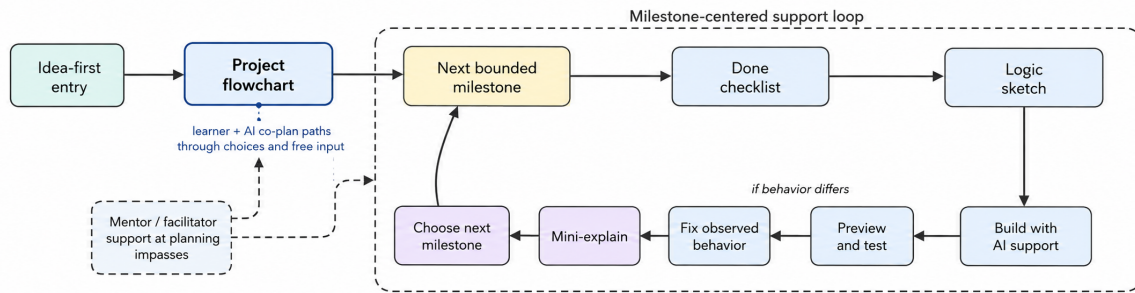
## 3 Goal-to-Milestone Framework

The Goal-to-Milestone framework is the paper’s central contribution. It is an interaction architecture for keeping AI support close to learner-owned project progression and metacognitive practice. As shown in Figure 1, the framework does not prohibit code generation. Instead, it places code generation inside a larger loop that begins with a learner’s idea, turns that idea into a revisable project flowchart, and repeatedly returns to bounded milestones. The central design question is not “What code should AI write?” but “What support helps the learner plan, monitor, understand, and complete the next meaningful step?”

The framework should be read as a sequence of learner-facing decisions, not as a hidden plan produced by the AI. First, idea-first entry protects the motivational source of student-initiated creative programming. The learner should be able to begin with a phrase, sketch, story, theme, or desired behavior rather than a fixed assignment template. The AI system’s first job is to reflect and clarify the idea: What kind of artifact does the learner imagine? Who might use it? What should someone be able to see, do, choose, change, or share?

Second, project flowchart construction turns a fuzzy idea into a visible project path. This is the most important step in the framework. The learner and AI reason backward from the imagined finished artifact and make design judgments about likely screens, states, data, interactions, feedback, and ordering. To reduce cognitive load, the assistant offers two to four possible paths phrased as choices rather than assignments, while leaving room for free input. The learner can select, combine, rename, reorder, skip, or replace path items. The output is not a full specification. It is a lightweight project flowchart: a revisable map of how the learner might complete the project step by step.

## Goal-to-Milestone Framework



**Figure 1:** The Goal-to-Milestone framework places code generation inside a broader support loop that includes project flowchart construction, criteria setting, logic sketching, preview-driven debugging, metacognitive reflection, and transfer to the next milestone.

Third, milestone planning turns the flowchart into an accessible first step. Each flowchart node becomes a candidate milestone only after the learner has helped name what visible behavior should change. A good first milestone should be easy to start, visibly connected to the desired project, and framed in terms of behavior the learner can inspect: “When I press Start, the first question appears” rather than “write the quiz logic.” This phrasing keeps implementation help tied to learner-selected criteria before code is generated.

Fourth, done checklists and logic sketches prepare the milestone before building. A done checklist states the observable criteria for success, such as “the Start button appears” and “pressing it shows one question.” These items are not grades; they are shared reference points for the learner, AI assistant, and mentor or facilitator. A logic sketch then makes the computational plan visible in pseudo-code, cards, a state diagram, or a natural-language sequence. Together, the checklist and sketch turn an implementation request into a planning-monitoring artifact.

Fifth, build, preview, fix, and mini-explain keep the loop tied to observed behavior. Preview-driven debugging begins from what the learner sees: What did you expect? What happened instead? Which checklist item failed? This shifts debugging from code replacement toward phenomenon-driven reasoning. A mini-explanation then closes the milestone with a concise link between the visible behavior and a transferable idea such as events, state, conditionals, or feedback. The learner can accept the explanation, ask a follow-up, or return to the project flowchart to choose the next milestone.

The framework therefore treats path and milestone planning as metacognitive practice. The learner is not only choosing a feature. The learner is practicing how to break down an imagined artifact, compare possible routes, judge feasibility, monitor progress, and revise a plan as understanding changes. Mentor and facilitator support enters at these metacognitive pressure points: when a learner is stuck judging a path, choosing a first milestone, revising the flowchart, or interpreting why a preview differs from the done checklist.

## 4 Design Scenarios

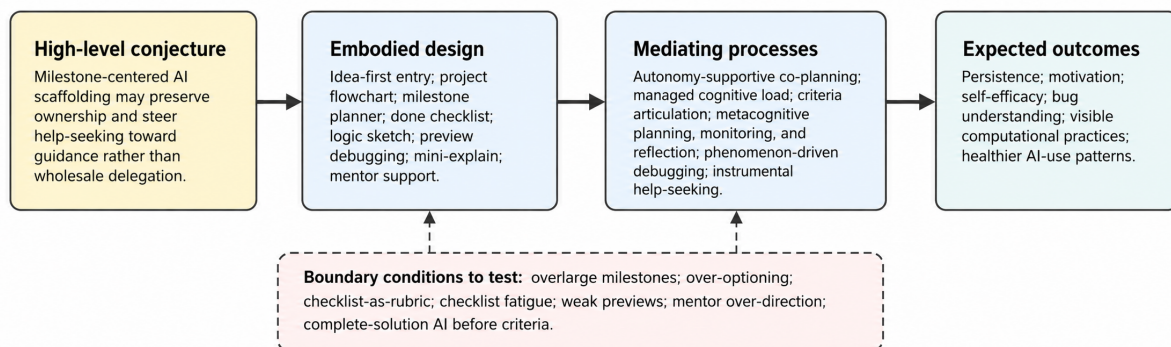
Consider a learner who begins with the idea, “I want to make a quiz game about our school.” A whole-project AI assistant might immediately generate a complete game. In the Goal-to-Milestone framework, the assistant first asks the learner to imagine the finished artifact: What will a player see first? How will they answer? What feedback should they get? The assistant then helps reverse-plan a project flowchart: start screen, one playable question, answer feedback, score, more questions, and styling. The learner chooses the playable-question path as an easy first milestone and edits it to match their intention: “When I press Start, the first

question appears with four answer choices.” After building, the preview shows the question but not the choices. Instead of replacing the code, the assistant asks which checklist item failed and points to the missing step in the logic sketch. The mini-explanation after the fix connects the visible behavior to state and event handling.

A second learner wants to make a mini-app for a club that lists upcoming volunteer tasks. The assistant asks what a club member should be able to do with the finished app, then helps the learner reverse-plan a flowchart: display tasks, add new tasks, mark tasks as done, filter by date, and share the app. The first milestone is behavioral: three volunteer tasks appear, each has a date, and the learner can add a new task in preview. When the learner gets stuck deciding whether storage should be part of the first milestone, a facilitator helps compare two paths: keep the first milestone visual and temporary, or expand it to include persistence. If the preview later shows new tasks disappearing after refresh, the assistant frames the bug as observed behavior and explains the distinction between temporary interface state and persistent storage. In both scenarios, the framework’s core work is not simply decomposition; it is co-constructed project planning made visible enough for the learner to inspect and revise.

## 5 Conjecture Map and Research Agenda

Conjecture mapping is useful for expressing what this framework claims and what it leaves for future study. In design-based research, conjecture maps connect high-level conjectures to embodied design elements, mediating processes, and learning outcomes (Sandoval, 2014). This structure is appropriate here because the framework is not yet an evaluated intervention. The central conjecture is that milestone-centered AI scaffolding may help preserve learner ownership and steer learners toward productive help-seeking because the interface repeatedly asks them to reason backward from a project idea, make design judgments visible in a flowchart, articulate success criteria, and compare expected with observed behavior before receiving broad implementation help.



**Figure 2:** A conjecture map for future research on milestone-centered AI support. The map states anticipated mediating processes and outcomes to investigate, not established causal effects.

As shown in Figure 2, the embodied design elements are expected to shape mediating processes rather than outcomes directly. Project flowcharts may support metacognitive planning; co-constructed paths may support autonomy while reducing open-ended planning load; done checklists may support criteria articulation and monitoring; preview-driven debugging may support monitoring of program behavior; mini-explanations may support reflection; and mentor visibility may support moments when learners are stuck making path decisions.

Future empirical work should test these conjectures in ways that respect hybrid learning settings. The most important measures are not only whether learners finish projects, but whether they revise flowcharts, articulate done criteria, compare expected and observed be-

havior, ask more inquiry-oriented questions, and use mentor or facilitator support at planning impasses.

## 6 Implications and Limitations

Student-facing AI coding tools need metacognitive progression architecture, not only stronger generation models. Tool builders and curriculum designers can treat project flowcharts, milestones, checklists, sketches, preview observations, and next-step bridges as reusable interface primitives for AI-supported creative programming. Mentor and facilitator support should also be designed around metacognitive need rather than constant control: the key role is to support learners when they are stuck making path judgments, choosing an appropriate first milestone, revising the flowchart, or interpreting why a preview differs from the done checklist.

This paper has three limitations. First, it offers a literature-grounded position and design framework, not empirical evidence of improved learning outcomes. Second, direct research on K–12 vibe coding is still emerging. Third, the framework could fail under predictable conditions: milestones may be too large, over-optional paths may narrow learner ideas, checklists may become rubrics, too many checklist prompts may make informal creative programming feel school-like, preview surfaces may be too weak to support debugging, mentors may over-direct, or AI responses may provide complete solutions before learners have articulated criteria.

## 7 Conclusion

AI coding tools invite computing education to revisit what kind of help young learners need when they make software. For K–12 creative programming, we argue that the most important design target is not simply faster code generation, but learner-owned progression from idea to project flowchart to milestone to next milestone. The Goal-to-Milestone framework reframes AI support around metacognitive planning, bounded steps, visible criteria, logic sketches, preview-driven debugging, and lightweight explanation. Its value now is not as a proven intervention, but as a concrete position: if AI is to support youth creative programming well, it should help learners keep deciding what they are trying to make happen next and why.

## References

- Brennan, K., & Resnick, M. (2012). New frameworks for studying and assessing the development of computational thinking. In *Proceedings of the 2012 Annual Meeting of the American Educational Research Association*.
- Deci, E. L., & Ryan, R. M. (2000). The “what” and “why” of goal pursuits: Human needs and the self-determination of behavior. *Psychological Inquiry*, 11(4), 227–268. [https://doi.org/10.1207/S15327965PLI1104\\_01](https://doi.org/10.1207/S15327965PLI1104_01)
- Druga, S., & Ko, A. J. (2025). Scratch Copilot: Supporting youth creative coding with AI. In *Proceedings of the 24th Interaction Design and Children* (pp. 140–153). ACM. <https://doi.org/10.1145/3713043.3727051>
- Flavell, J. H. (1979). Metacognition and cognitive monitoring: A new area of cognitive-developmental inquiry. *American Psychologist*, 34(10), 906–911. <https://doi.org/10.1037/0003-066X.34.10.906>
- Gardner, T., Leonard, H. C., Waite, J., & Sentance, S. (2022). What do we know about computing education for K–12 in non-formal settings? A systematic literature review of recent research. In *Proceedings of the 2022 ACM Conference on International Computing Education Research* (pp. 264–281). ACM. <https://doi.org/10.1145/3501385.3543960>
- Kafai, Y. B., & Burke, Q. (2014). *Connected code: Why children need to learn programming*. MIT Press.
- Kazemitabaar, M., Chow, J., Ma, C. K. T., Ericson, B. J., Weintrop, D., & Grossman, T. (2023). Studying the effect of AI code generators on supporting novice learners in introductory programming. In *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems* (pp. 1–23). ACM. <https://doi.org/10.1145/3544548.3580919>
- Kazemitabaar, M., Ye, R., Wang, X., Henley, A. Z., Denny, P., Craig, M., & Grossman, T. (2024). CodeAid: Evaluating a classroom deployment of an LLM-based programming assistant that balances student and educa-

- tor needs. In *Proceedings of the CHI Conference on Human Factors in Computing Systems*. ACM. <https://doi.org/10.1145/3613904.3642773>
- Papert, S. (1980). *Mindstorms: Children, computers, and powerful ideas*. Basic Books.
- Sandoval, W. (2014). Conjecture mapping: An approach to systematic educational design research. *Journal of the Learning Sciences*, 23(1), 18–36. <https://doi.org/10.1080/10508406.2013.778204>
- Sentance, S., & Waite, J. (2017). PRIMM: Exploring pedagogical approaches for teaching text-based programming in school. In *Proceedings of the 12th Workshop on Primary and Secondary Computing Education* (pp. 113–114). ACM. <https://doi.org/10.1145/3137065.3137084>
- Shin, Y., Jung, J., Zumbach, J., & Yi, E. (2023). The effects of worked-out example and metacognitive scaffolding on problem-solving programming. *Journal of Educational Computing Research*, 61(6), 1312–1331. <https://doi.org/10.1177/07356331231174454>
- Si, J. J., Li, D., Wang, Q., Liu, A. W., Lin, X., Zhu, Y., Zhou, X., Wang, A. Y., & Wang, K. Y. (2026). Exploring the impacts and challenges of vibe coding paradigm to children's programming learning and practices. In *Proceedings of the 2026 CHI Conference on Human Factors in Computing Systems*. ACM. <https://doi.org/10.1145/3772318.3790366>
- Si, Y., Qi, K., Li, D., Shi, H., & Zhang, J. (2025). *Stitch: Step-by-step LLM guided tutoring for Scratch*. arXiv. <https://arxiv.org/abs/2510.26634>
- Sweller, J. (1988). Cognitive load during problem solving: Effects on learning. *Cognitive Science*, 12(2), 257–285. [https://doi.org/10.1016/0364-0213\(88\)90023-7](https://doi.org/10.1016/0364-0213(88)90023-7)
- Thorgeirsson, S., Weidmann, T. B., & Su, Z. (2026). Computer science achievement and writing skills predict vibe coding proficiency. In *Proceedings of the 2026 CHI Conference on Human Factors in Computing Systems*. ACM. <https://doi.org/10.1145/3772318.3791666>
- Wood, D., Bruner, J. S., & Ross, G. (1976). The role of tutoring in problem solving. *Journal of Child Psychology and Psychiatry*, 17(2), 89–100. <https://doi.org/10.1111/j.1469-7610.1976.tb00381.x>
- Zimmerman, B. J. (2002). Becoming a self-regulated learner: An overview. *Theory Into Practice*, 41(2), 64–70. [https://doi.org/10.1207/S15430421TIP4102\\_2](https://doi.org/10.1207/S15430421TIP4102_2)